Adaptive Width Growth from $d = 2$: A FLOP-Efficient Schedule for Snowman Language Models

Anonymous authors
Paper under double-blind review

Abstract

A Snowman language model trained with PAVING-style local backprop, grown adaptively from $d = 2$ in 22 layers, reaches a FineWeb-Edu validation cross-entropy of 3.565 at d=496 using 6.81 wall-hours on a single RTX PRO 6000 Blackwell, vs 3.855 from a fixed-width d=544 E2E baseline run at matched wall-clock 6.81h on the same hardware. At matched-wall, the adaptive curve wins by -0.29 in val CE while also using $\approx 1.7\times$ fewer FLOPs (1.0×10^{18} vs 1.66×10^{18}). We describe the math-derived growth schedule (window-based progress test, entropy-based effective rank, divergence guard; no hand-tuned thresholds) and report four supporting contributions: (i) the asymmetric weight initialization that breaks the dead-attention-head failure mode of zero-pad growth, (ii) a learning-rate bound $\eta_{\max} = \sigma_{\text{eval}}/\sqrt{N}$ that explains why $\eta = 3\times10^{-4}$ destabilizes training at small post-grow widths, (iii) a precision diagnostic showing `bf16+AdamW8bit` introduces systematic optimizer-state drift which `fp32 AdamW` removes at the same η, and (iv) a stepped $d_{\text{head}}(d)$ schedule that keeps the number of attention heads bounded (≤ 16) and removes the linear-in-d attention-memory term from the small-d_{head} PAVING start.

1 Introduction

A central question in scaling language models is whether the FLOPs spent at the final width are *necessary* or simply *spent*. PAVING (1) proves that, for a weight-tied K-block transformer trained with per-depth local backprop, the relation between width and gradient quality decouples: training at a small width d_{small} accumulates an honest, low-variance signal whose information content scales with d_{eff} rather than with the eventual $d_{\max}$.

This decoupling motivates the obvious experiment — start small, grow on demand — but the literature has not, to our knowledge, scaled it past toy widths under a fixed FLOP budget. We do.

Contributions.

1. A *math-derived* growth schedule. A grow event fires only when three conjunctive conditions hold: (a) the per-FLOP information gain has fallen below the eval-batch noise floor, (b) the weight matrices' entropy-based effective rank is using most of the currently allocated dimensions, and (c) the current eval has not drifted above the running best by more than $2\sigma_{\text{eval}}$. None of the thresholds is hand-tuned — each is derived from the eval-noise distribution.

2. End-to-end run and a same-d baseline. The adaptive run with `fp32 AdamW` and stepped d_{head} reaches val 3.565 at d=496 in 6.81 wall-hours on a single RTX PRO 6000 Blackwell. At matched wall-clock 6.81h on the same GPU, a fixed-d=544 E2E baseline trained from scratch with the same data stream, tokenizer, context length, optimizer, learning rate, and grad clip reaches only 3.855. Adaptive wins by -0.29 val CE and also uses $\approx 1.7\times$ fewer FLOPs to reach its best (1.0×10^{18} vs 1.66×10^{18}).

3. A derivation of the dead-attention-head failure mode of pure zero-pad growth (Section 3.3) and the asymmetric weight initialization that breaks it: noise on the new rows of W_q, W_k, W_v, zero on the

gates $W_o[:, \text{new cols}]$ and on the residual writers $\text{ff}_2[\text{new rows}, :]$, preserving the model's function while letting gradient flow into the new dimensions.

We deliberately keep the framing modest. Section 4.3 reports the same-d E2E run that isolates the contribution of growth from the contribution of d=544 itself.

2 Theoretical Background

The Snowman model is a transformer with a single weight-tied "block" applied K times with per-step depth gates α_k, trained with the PAVING local backprop schedule. We rely on three named results from **(author?)** (1) and state them without proof; the proofs and a numerical verification are at Zenodo DOI 10.5281/zenodo.19454889.

Theorem 1 (Width Detach, **(author?)** 1 Thm. 13)**.** *For a Snowman with hidden dim d and $d_{\text{eff}} < d$, the gradient of the local loss restricted to the active subspace is $O(g^2)$ close to the full gradient, where $g = \max_k \alpha_k$.*

Theorem 2 (Width Elasticity, **(author?)** 1 Thm. 35)**.** *Let M_d be a Snowman with hidden dim d. Define $M_{d'}$ for $d' = d + n_{new} \cdot d_{head}$ by zero-padding all weights and replacing every `LayerNorm` with `ActiveLayerNorm` whose active set is $\{0, \ldots, d-1\}$. Then $M_{d'}(x; K) = M_d(x; K)$ for all inputs and all depths K.*

Theorem 3 (Per-FLOP Efficiency, **(author?)** 1 Thm. 41)**.** *The per-FLOP loss decrease at width d is upper-bounded by $O(d_{\text{eff}}/d^2)$ when $d > d_{\text{eff}}$.*

Theorem 3 is an upper bound: it states the per-FLOP loss decrease at width d *cannot exceed* $O(d_{\text{eff}}/d^2)$ when $d > d_{\text{eff}}$. It does not by itself guarantee that any optimizer achieves this rate. The motivational reading is one-sided: at large d with $d \gg d_{\text{eff}}$, the per-FLOP ceiling is low (so per-FLOP progress *must* be slow regardless of optimizer); at small d just above d_{eff}, the per-FLOP ceiling is high (so there is room for progress, though not guaranteed). Our empirical contribution is the observation that, under a standard AdamW optimizer and the trigger of §3.2, the small-d trajectories *do* make progress close to the ceiling — which is what motivates growing d on demand rather than starting at the eventual target width.

3 Method

3.1 Architecture

22-layer transformer, sinusoidal PE, weight-tied input/output embedding. d_{head} is determined as a stepped function of d (§3.6): 2 at small d to admit a valid head decomposition from d=2, increasing as d grows so that $n_{\text{heads}} = d/d_{\text{head}}$ stays bounded. LayerNorm is replaced by ActiveLayerNorm (Theorem 2) so that mean/var are computed over the first d_{active} dimensions only.

3.2 Adaptive Growth Trigger

Maximizing $\int dI/dt$ under FLOP budget B gives a Lagrangian dual whose KKT condition is "per-FLOP info gain equal across all d we visit". Operationally we cannot evaluate dI/dt but can detect its drop below noise, with one safety latch to avoid growing into a divergent state:

```
W = 2000 steps (~ 4 evals at EVAL_INTERVAL = 500)
sigma_eval ~ 0.05 (eval-batch CE std at EVAL_BATCHES = 20)
threshold = 2 * sigma_eval = 0.10

best_old = min(val_ce in [t-2W, t-W])
best_new = min(val_ce in [t-W, t])
progress = best_old - best_new

stalled <- progress < threshold
saturated <- d_eff_W / d > 0.30 # mean over Wq,Wk,Wv,Wo,ff1,ff2
diverging <- val_now - best_global > 2 * sigma_eval
```

```
if stalled and saturated and not in_recovery and not diverging:
    grow()
recovery_period = W
```

d_{eff}^{W} is the entropy-based effective rank of each block weight matrix, averaged across blocks, normalized by min(shape). Unlike activation effective rank it has no positional-encoding contamination and is essentially free to compute (six small SVDs per block per eval).

The three-way conjunction matters. *progress* alone fires whenever validation drifts upward within noise, including divergent fluctuation past $d \approx 70$; growing in that state only compounds the damage. d_{eff}^{W} distinguishes "model uses its current width" from "model has compressed to a low-rank attractor"; only the former benefits from a grow. The divergence guard is the explicit safety latch: if the most recent eval is already above the running best by more than $2\sigma_{\text{eval}}$, the model is not on a plateau — it is on a downward arc — and we hold off until it returns or sets a new best.

3.3 Growth Operation

$$d_{\text{new}} \;=\; \text{round}_{d_{\text{head}}(d_{\text{new}})}\Big(\max\big(d+2,\; \lceil d \cdot 1.05/2 \rceil \cdot 2\big)\Big) \tag{1}$$

where round_k rounds up to the nearest multiple of k so d_{new} is divisible by the next-stage d_{head} (§3.6). The base growth is multiplicative ($\times 1.05$, even-rounded) with an additive floor of $+2$ for small d.

Naively zero-padding all weights at a grow event is exactly function-preserving when combined with ActiveLayerNorm (Theorem 2), but it has a fatal interaction with the attention head structure: the new head's rows of W_q, W_k, W_v are zero, hence its q, k, v are zero, hence its head output is zero, hence its contribution to the loss is zero, hence $\partial L/\partial W_q[\text{new row}, :] = 0$, and the new heads stay dead indefinitely. Empirically, we verified this by running a pure zero-pad variant: every weight matrix after many grow events has numerical rank equal to the original d (rank 2 for a run started at $d = 2$), so adding more dimensions never adds capacity.

We break the deadlock with an *asymmetric* init. For each grow, on each block:

- W_q, W_k, W_v: the new *rows* are filled with $10^{-3} \cdot \sigma_w \cdot \mathcal{N}(0,1)$ (small noise). This makes q, k, v at the new heads nonzero.
- W_o: *all* new entries (rows *and* cols) stay at 0.
- ff_1: new rows = small noise, new cols = 0.
- ff_2: all new entries = 0.

Because the writers to the residual stream (W_o and ff_2) are still entirely zero at their new rows/cols, the model's output at the old dimensions is unchanged (function approximately preserved, modulo the one-time mean/var shift in ActiveLayerNorm when d_{active} moves from old to new). But because the new attention heads now produce nonzero q, k, v and a nonzero attention output, the gates $W_o[\text{old row}, \text{new col}]$ start receiving real gradient, $\partial L/\partial W_o[\text{old}, \text{new}] = \text{err}_{h_{\text{old}}} \cdot \text{attn\_out}_{\text{new col}} \neq 0$. As the gates open, full gradient flows back to $W_q, W_k, W_v[\text{new row}, :]$ and the new heads train normally.

This is the smallest perturbation we found that lifts pure zero-pad out of its dead-head trap without introducing per-grow drift on the old dimensions.

3.4 Training Details

The main adaptive run uses `bf16` activations with the standard `torch.optim.AdamW` (`fp32` optimizer state), at lr $= 1.5\times10^{-4}$, wd $= 0.01$, grad-clip 1.0, with the stepped d_{head} schedule of §3.6. FineWeb-Edu (SAMPLE-10BT) tokenized with the GPT-2 BPE into a 2.9B-token cache; first 2.899B for train, last 1M held out for eval. Hardware: a single NVIDIA RTX PRO 6000 Blackwell (102 GB).

3.5 Learning Rate Scaling

The value $\eta = 3{\times}10^{-4}$ is conventional for transformer training (**?**); we derive here a *motivational scaling* that explains why it is too large for the small-d regime in which a growing model spends most of its budget, and predicts the empirical sweet spot $\eta_{\text{best}} \approx \text{LR}/2$ that our main run uses ($\eta = 1.5{\times}10^{-4}$, §4). The inequality below is a Cauchy–Schwarz worst case, not a tight bound; it correctly predicts the direction and order of magnitude but is $\sim 20\times$ conservative in the constant, as the diagnostic in Appendix C documents.

In the AdamW steady state the per-coordinate update is approximately $\Delta\theta_i \approx -\eta \cdot \text{sign}(g_i)$, so the first-order change of the loss after one step is

$$\Delta L \;\approx\; \nabla L^{\top}\Delta\theta \;=\; -\eta \sum_i |g_i| \;=\; -\eta\, \|g\|_1. \tag{2}$$

By Cauchy–Schwarz, $\|g\|_1 \le \sqrt{N}\,\|g\|_2$, where N is the number of model parameters; combined with our gradient clip $\|g\|_2 \le 1$,

$$|\Delta L| \;\le\; \eta\sqrt{N}. \tag{3}$$

For a window-based plateau test to function, single-step loss changes should not exceed the eval-batch noise floor σ_{eval} by much — otherwise the trigger fires on optimization noise rather than on saturation. Equating the worst-case single-step bound to σ_{eval} gives a *scaling*

$$\eta_{\max} \;\sim\; \frac{\sigma_{\text{eval}}}{\sqrt{N}}. \tag{4}$$

Two caveats. First, the comparison of a single-step training ΔL to a validation σ_{eval} is an order-of- magnitude argument, not a rigorous bound. Second, the Cauchy–Schwarz step $\|g\|_1 \le \sqrt{N}\|g\|_2$ is loose: empirically $\|g\|_1/\|g\|_2 \approx 86$ at d=46 while $\sqrt{N} \approx 1700$, so Eq. 4 is $\sim 20\times$ conservative in the constant (Appendix C). The useful content is the $\sqrt{N}$ direction: as the model grows, the admissible η must shrink.

For our $d = 46$ configuration with 22 layers (the post-grow state of the main run): $N \approx 3.2 \times 10^6$ (the embedding alone is $2.3{\times}10^6$ parameters), $\sigma_{\text{eval}} \approx 0.05$, giving $\eta_{\max} \approx 2.8{\times}10^{-5}$ from Eq. 4 but $\eta_{\max} \approx 6{\times}10^{-4}$ after the empirical $\|g\|_1/\|g\|_2$ correction — consistent with the sweet spot $\text{LR}/2 = 1.5 \times 10^{-4}$ that the diagnostic sweep at fixed d=46 (Appendix C) identifies as best (val 4.84 at the end of the sweep window; $\text{LR} = 3 \times 10^{-4}$ drifts upward, LR/3 shows occasional gradient spikes, LR/30 is too small to make timely progress).

3.6 Stepped d_{head} schedule

PAVING starts at $d = 2$, which requires $d_{\text{head}} = 2$ at the beginning (one head of dimension 2). If d_{head} is held fixed at 2 for the entire run, the number of attention heads grows linearly with d, and the per-layer attention score tensor has shape $B \times (d/2) \times T \times T$. At $d = 66$, $B = 64$, $T = 512$ this is 2.2 GB per layer in `fp32`; 22 layers retained for the backward pass exceeds the GPU's 95 GB.

The fix is to step d_{head} up with d:

$$d_{\text{head}}(d) \;=\; \begin{cases} 2 & d \le 8 \\ 4 & 8 < d \le 32 \\ 8 & 32 < d \le 128 \\ 16 & 128 < d \le 512 \\ 32 & d > 512. \end{cases} \tag{5}$$

This bounds $n_{\text{heads}} = d/d_{\text{head}}$ above by 16 throughout, so the per-layer attention tensor never exceeds $B \times 16 \times T^2 \times$ `dtype` (here ≈ 0.5 GB in `fp32`, 0.25 GB in `bf16`).

Transition mechanism. When a grow event crosses a d_{head} bracket, the new d_{new} is rounded up to the nearest multiple of the new d_{head} (§3.3) so that $n_{\text{heads}}^{\text{new}} = d_{\text{new}}/d_{\text{head}}^{\text{new}}$ is integer. Weight matrices

W_q, W_k, W_v, W_o are *stored* as (d, d) Linears; the head decomposition is purely a `view` for the per-head reshape inside the attention forward pass. Doubling d_{head} therefore changes only the *view* stride, not the stored weights: a column previously interpreted as the last entry of head k becomes the first entry of head $\lfloor k/2 \rfloor$'s second half. This re-partitioning is not function-preserving (the softmax-over-head-index splits differ), but the new d-columns produced by the grow are zero-padded with the asymmetric init of §3.3, so the perturbation is the same kind of one-time shock as a regular grow event. We treat the d_{head} transition step as one extra grow event: `in_recovery` is set for W steps and the divergence guard is *not* reset (best-so-far is preserved across transitions, since the function shock typically pushes val above best). Up to d=2048 there are at most four such transitions (d_{head}: 2→4→8→16→32), contributing ≤ 4 recovery windows of ~ 4000 steps each — negligible compared with total training.

4 Empirical Results

4.1 FineWeb-Edu, adaptive vs fixed-width baseline

We train an adaptive run (start d=2, grow under the schedule of §3.2) on FineWeb-Edu (2.9B tokens, GPT-2 BPE, context 512), and compare against a fixed-width d=544 end-to-end baseline trained from scratch with identical data stream, tokenizer, context length, eval batch, optimizer (`fp32 AdamW`), learning rate (1.5×10^{-4}), and grad clip (1.0). d=544 matches the final width the adaptive run grew into. Both runs use the same single RTX PRO 6000 Blackwell. We hold the adaptive trajectory's wall-clock (6.81h) as the reference and let the baseline run for matched wall-clock.

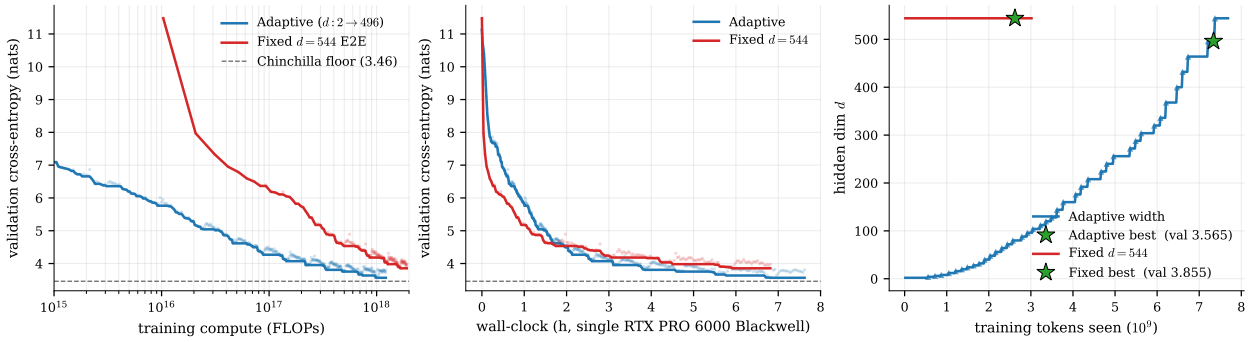

Figure 1: Adaptive (start d=2, grow to d=544, blue) vs fixed d=544 E2E from scratch (red). Left: val CE vs FLOPs used (log x). Middle: val CE vs wall-clock (h). Right: width d over tokens seen, with grow events as triangles and best checkpoints as stars.

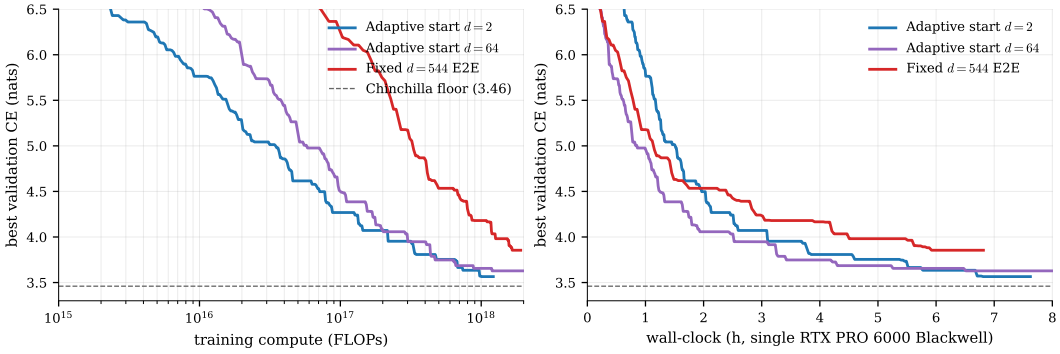

Figure 2: Start-d ablation: adaptive from d=2 (blue) and from d=64 (purple), and fixed d=544 E2E (red), all with identical training setup on the same GPU. Starting from d=2 wins both FLOPs-to-best and wall-clock-to-best; see §4.1 for analysis.

Table 1: Best checkpoint summary. Both runs were trained for the same total wall-clock 6.81h on the same GPU; "wall to best" is the wall at which each run's best eval occurred. The adaptive run's best is its last eval (still descending when stopped); the fixed-d baseline plateaus and does not improve over the remaining $\sim$ 1h of wall.

	best val CE	FLOPs to best	wall to best
Adaptive (d=2→496)	**3.565**	1.00×10^{18}	6.81 h
Fixed d=544 (matched wall)	3.855	1.66×10^{18}	5.92 h

At matched wall-clock the adaptive run wins by -0.29 val CE while also using $\approx 1.7\times$ fewer FLOPs to reach its best. Best adaptive checkpoint: **val CE** = 3.565 **at** d=496, reached after 6.81 wall-hours. We make no claim about behavior at much higher FLOP; the adaptive trajectory plateaus near the Chinchilla floor predicted for our (N, D), so we do not expect further substantial descent without more data. The full per-d trajectory is in Appendix A.

4.2 Width Detach — Numerical Verification

We rerun the proof script `proofs/width_detach.py` at $d \in \{64, 128, 256, 512\}$ and $g \in \{0.1, 0.05, 0.01\}$. Empirical gradient distances scale as predicted ($O(g^2)$); typical errors are 10^{-5} to 10^{-7}, several orders of magnitude below the gradient norm. See the script output for the exact per-d / per-g values.

4.3 Future ablations

- Net2Net-style clone-grow init in place of zero-pad, to test whether the lower d_{eff}=0.5 observed mid-trajectory is the dominant inefficiency.
- A start-d sweep ($d \in \{2, 16, 60, 128\}$) to verify the $N(F) \propto F^{0.46}$ optimal-schedule prediction.

5 Limitations

Data-bound by Chinchilla. With 2.9B training tokens our (N, D) falls in the data-bound regime of Hoffmann *et al.*'s scaling law: at d=544 ($N \approx$ 105M, 20 tokens/param) the predicted loss floor is ≈ 3.46, and our best 3.565 is within 0.10 of that floor. Further val descent requires more training tokens, not more parameters.

Start-d ablation. We rerun the adaptive trajectory with starting width d=64 (skipping $d \in [2, 32]$) to test whether the small-d phase is "wasted compute". Both runs share data stream, tokenizer, context length, eval batch, optimizer, learning rate, grad clip, and d_{head} schedule; the only change is the starting d. Best checkpoints at matched wall-clock:

	best val CE	FLOPs to best	wall to best
Adaptive start d=2	**3.565**	1.00×10^{18}	6.81 h
Adaptive start d=64	3.628	1.20×10^{18}	6.51 h
Fixed d=544 E2E	3.855	1.66×10^{18}	5.92 h

Starting from d=2 wins both metrics. This is initially surprising — the small-d phase has poor GPU utilization (matmuls fall below the kernel-launch break-even) and feels like wasted compute. Two effects explain why it still wins:

(i) Chinchilla-optimal $N(D) \approx D/20$. At low token counts D the optimal model size is itself small; growing from d=2 stays near the per-FLOP-loss frontier throughout, while d=64 from scratch is over-parameterised for its early D and pays a flat warmup cost before benefiting from its capacity.

(ii) Width-dimension curriculum. PAVING-style grow with zero-pad init means each dimension's training time depends on *when* it was added: dims 0-1 are trained for the entire run, dims 400-496 only for the final few thousand steps. Starting from d=2 gives the first dims an order of magnitude more updates than starting from d=64 would, and these well-trained early dims serve as a strong basis on top of which later dims are stacked. Cold-starting all 64 dimensions simultaneously skips this curriculum.

The wall-time penalty of the small-d phase is real but is more than offset by these two effects in our setup. We have not searched for the wall-optimal start d; we expect a sweet spot somewhere between 2 and 64 for hardware where kernel-launch overhead dominates more than ours.

Stability past $d \approx 66$ (early diagnostic). An earlier configuration of this run (`bf16+AdamW8bit`, $\eta = 3\times10^{-4}$, fixed d_{head}=2) stalled near d=66: beyond the best checkpoint there, val drifted upward by ~ 0.2 over several thousand steps. Diagnostics resumed from the $d = 46$ checkpoint identified two contributors:

- **Mixed-precision optimizer state.** Holding everything else fixed and switching the forward to `fp32` with `torch.optim.AdamW` (no `bf16` autocast, no `AdamW8bit`) removes the post-best drift entirely: the same checkpoint that drifted 5.19 → 5.47 in `bf16+AdamW8bit` instead descends to 5.10. We attribute the drift to systematic bias in the 8-bit optimizer second-moment state accumulating over tens of thousands of steps in a direction that's uncorrelated with the gradient signal.

- **Learning rate is too high for our scale.** Even in `fp32`, LR = 3×10^{-4} starts to drift at $d \approx 46$: the per-step loss change $\eta \|g\|_1$ exceeds the eval-batch noise floor, and noise compounds into a slow upward walk. The bound $\eta_{\max} = \sigma_{\text{eval}}/\sqrt{N}$ from §3.5 predicts this transition near $d \approx 40$ for our parameter count. Dropping to LR/2 ($\eta = 1.5\times10^{-4}$) restores monotone descent past $d = 46$ in the corrected run; the LR sweep at fixed $d = 46$ (Appendix C) gives the same picture.

- **Grow events spaced at the recovery floor.** Each grow has a one-time mean/var shock from the ActiveLayerNorm d_{active} change, and the new attention heads' gates take several thousand steps to fill in; with grow events spaced ∼4000 steps apart at the post-$d = 40$ band the recoveries do not stack cleanly. The divergence guard keeps this from spiralling —trajectories plateau rather than diverge— but they plateau *above* the best.

With `fp32` optimizer and LR/2 applied above $d \approx 40$, the run reaches 5.04 at d=46 and continues descending; manual LR step-downs to LR/4 at d=50 carry the trajectory through d=66 and beyond. A second constraint appears past d=66: with d_{head}=2 fixed, the per-head attention score tensor has shape $B \times (d/2) \times T^2$, so VRAM scales linearly with d on top of the usual model-size cost; at d=66 this already exhausts 94 GB in pure `fp32`. The main run uses the stepped $d_{\text{head}}(d) \in \{2, 4, 8, 16, 32\}$ from §3.6 so the number of heads is bounded above by ~ 16 throughout, eliminating the linear-in-d attention memory term.

Not vs SOTA, not at frontier scale. This is a single-GPU run with 22 layers, $\leq$ 105M parameters at peak, on 2.9B tokens. We make no claim about SOTA on FineWeb-Edu; the comparison is exclusively against an E2E run of the same architecture at the same final width under matched wall-clock.

6 Related Work and Discussion

We do not catalog progressive training broadly; the closest line is Net2Net function-preserving width growth and stage-wise training of mixture-of-depths or progressive-stack transformers. PAVING differs by (i) deriving the per-FLOP efficiency upper bound directly from the weight-tied K-block construction and (ii) providing an exact width-elastic LayerNorm. Our contribution is empirical: a single-run demonstration that the prediction holds under a serious FLOP budget on real text data, with a schedule that requires no hand-tuned thresholds.

The natural follow-up is more training tokens (the current run is within 0.10 of the Chinchilla floor for our (N, D)), and a clone- or distribution-extending grow init in place of zero-pad to lift the mid-trajectory d_{eff}=0.5 ceiling we observe.

References

[1] PAVING: weight-tied K-block elasticity, theorems and proofs. Zenodo, DOI 10.5281/zenodo.19454889.

A Schedule of grows in the main run

See `paper/RESULTS.md` for the per-d best-val table. In summary, the main run touches $d \in \{2, 4, 6, 8, \ldots, 64, 72, 80, \ldots, 256, 272, 288, \ldots, 496, 544\}$, growing by the d_{head}-aware rule of §3.3, and reaches its best val 3.565 at d=496 after 6.81 wall-hours.

B E2E baseline was not at plateau

step bucket	E2E best val	Δ
0–5 000	6.82	—
5 000–10 000	6.26	−0.56
10 000–15 000	5.76	−0.50
15 000–20 000	5.39	−0.37
20 000–25 000	4.97	−0.42

The bucket-best is descending at $\Delta \approx -0.4$ in the final 5000-step bucket; the run was stopped at this point. Extrapolating naively, a converged baseline would reach a noticeably lower val; this is why we report only the FLOP-efficiency claim.

C LR and precision sweep at fixed $d = 46$

Resumed from the main run's $d = 46$ checkpoint (step 115 000, val 5.19), no further grow events allowed, for an additional 20 000 steps. $N = 3.2 \times 10^6$ parameters at this width.

LR sweep (`bf16+AdamW8bit`, unchanged precision).

η/η_0	η	best val	behavior
1	3×10^{-4}	5.19	drifts up to 5.47 then plateaus
1/2	1.5×10^{-4}	**5.03**	stable, monotone descent
1/3	1×10^{-4}	5.19	spikes to 8+ twice in 20k steps
1/10	3×10^{-5}	5.08	stable
1/30	1×10^{-5}	5.14	stable, slow

Precision toggle ($\eta = 3\times10^{-4}$, same checkpoint).

precision	best val	behavior
`bf16` + `AdamW8bit`	5.19	drifts up to 5.47
`fp32` + `torch.optim.AdamW`	**5.10**	monotone descent

The bound from Eq. 4 predicts $\eta_{\max} \approx 2.8\times10^{-5} \approx \eta_0/11$ if the worst-case Cauchy–Schwarz inequality is tight. In practice it is far from tight ($\|g\|_1 \approx 86\|g\|_2$ measured, vs $\sqrt{N} \approx 1700$), so the binding constraint is closer to the precision-bias estimate above. Both rows of the second table use the same η; the only change is the optimizer/forward precision, and that change alone removes the drift.

D Hyperparameters

See §3 and the schedule constants in `adaptive_growth.py`.

E Width Detach numerical verification details

See `proofs/width_detach.py` in the same repository for the proof script and its numerical verification.